

Imp Language Extension in Haskell

Calinov Cosmin

March 7, 2026

1 Introduction

2 Definition

3 Abstract Syntax Tree

3.1 Formal Grammar

What programs look like

```
 $\langle Exp \rangle ::= \langle Integer \rangle \mid \langle Variable \rangle$   
|  $\langle Exp \rangle \text{ '+' } \langle Exp \rangle \mid \langle Exp \rangle \text{ '*' } \langle Exp \rangle$   
|  $\langle Exp \rangle \text{ '-' } \langle Exp \rangle \mid \langle Exp \rangle \text{ '/' } \langle Exp \rangle$   
|  $\langle Exp \rangle \text{ '%' } \langle Exp \rangle \mid \langle Exp \rangle \text{ '^' } \langle Exp \rangle$   
| true | false  
|  $\langle Exp \rangle \text{ '==' } \langle Exp \rangle \mid \langle Exp \rangle \text{ '<=' } \langle Exp \rangle$   
|  $\langle Exp \rangle \text{ '>=' } \langle Exp \rangle \mid \langle Exp \rangle \text{ '<' } \langle Exp \rangle$   
|  $\langle Exp \rangle \text{ '>' } \langle Exp \rangle \mid \text{not } \langle Exp \rangle$   
|  $\langle Exp \rangle \text{ '&\&' } \langle Exp \rangle \mid \langle Exp \rangle \text{ '||' } \langle Exp \rangle$   
  
 $\langle Stmt \rangle ::= \text{skip}$   
|  $\langle Variable \rangle \text{ ':=' } \langle Exp \rangle$   
|  $\langle Stmt \rangle \text{ ';' } \langle Stmt \rangle$   
| if  $\langle Exp \rangle$  then  $\langle Stmt \rangle$  else  $\langle Stmt \rangle$   
| while  $\langle Exp \rangle$  do  $\langle Stmt \rangle$   
| {  $\langle Stmt \rangle$  }  
| par [  $\langle Stmt \rangle$  ]
```

3.2 Data Types

Common Data Types

```
1 newtype Variable = Var String deriving (Eq,  
2 Ord, Show)
```

```

3      data Value = VInt Int | VBool Bool deriving
4                (Eq, Show)
5
6      newtype MyState = List [(Variable, Value)]
7                        deriving (Eq, Show)

```

Expression

```

1      data Exp = EVar Variable
2                | EInt Int
3                | EAdd Exp Exp
4                | EMul Exp Exp
5                | ESub Exp Exp
6                | EDiv Exp Exp
7                | EMod Exp Exp
8                | EPow Exp Exp
9                | BExp Bool
10               | BEq Exp Exp
11               | BLt Exp Exp
12               | BGt Exp Exp
13               | BLte Exp Exp
14               | BGte Exp Exp
15               | BAnd Exp Exp
16               | BOr Exp Exp
17               | BNot Exp
18      deriving Show

```

Statement

```

1      data Stmt = Assign Variable Exp -- equals
2                operator
3                | Seq Stmt Stmt -- sequence 2
4                  statements
5                | If Exp Stmt Stmt -- last
6                  statement for else branch
7                | While Exp Stmt
8                | Skip
9                | Block [Stmt]
10               | Par [Stmt]
11      deriving Show

```

4 Formal Semantics

What programs do

4.1 Skip

$$\overline{\langle \text{skip}, \sigma \rangle \Downarrow \sigma}$$

4.2 Assignment

$$\frac{\langle e, \sigma \rangle \Downarrow v}{\langle x := e, \sigma \rangle \Downarrow \sigma[x \mapsto v]}$$

4.3 Sequencing

$$\frac{\langle S_1, \sigma \rangle \Downarrow \sigma' \quad \langle S_2, \sigma' \rangle \Downarrow \sigma''}{\langle S_1; S_2, \sigma \rangle \Downarrow \sigma''}$$

4.4 While

True

$$\frac{\langle b, \sigma \rangle \Downarrow \text{true} \quad \langle S, \sigma \rangle \Downarrow \sigma' \quad \langle \text{while } b \text{ do } S, \sigma' \rangle \Downarrow \sigma''}{\langle \text{while } b \text{ do } S, \sigma \rangle \Downarrow \sigma''}$$

False

$$\frac{\langle b, \sigma \rangle \Downarrow \text{false}}{\langle \text{while } b \text{ do } S, \sigma \rangle \Downarrow \sigma}$$

5 Property-based Testing using QuickCheck

5.1 Skip doesn't modify the state of the program

Skip statement's usefulness is shown in this property.

$$\text{stmt}(\sigma, \text{Skip}) = \sigma$$

5.2 Sequential execution is compositional

A staple for all programming languages that are deterministic.

$$\text{stmt}(\sigma, \text{Seq } s_1 s_2) = \text{stmt}(\text{stmt}(\sigma, s_1), s_2)$$

5.3 Assigning a variable and then getting its value returns the assigned value

Basic testing for state's functionality.

$$\text{get}(\text{stmt}(\sigma, \text{Assign } x \ v), x) = v$$

5.4 Sequencing statements is associative

Doesn't matter in which order you run the operation, the end result is the same.

$$\text{stmt}(\sigma, \text{Seq}(\text{Seq}(s_1, s_2), s_3)) = \text{stmt}(\sigma, \text{Seq}(s_1, \text{Seq}(s_2, s_3)))$$

5.5 Program execution is deterministic

This property illustrates that the rules of the language are consistent i.e. running the same statement at two different times doesn't change it's outcome. This is especially useful when it comes to the Par statement, which is specifically designed to be deterministic. Other possible implementations could have chosen more stochastic approaches, the test failing in those particular cases. Another challenge was that While loops could run indefinitely. This was caused by the fact that the generated condition of the While loop would sometimes evaluate to True, thus running forever. I chose the simplest fix: I removed the the While statement from the Arbitrary instance of Stmt. The other possible fix would have been to add a "gas" limit for every evaluation of each statement, but this would have required to change the the signature and consequently a significant part of the evaluation function's logic.

$$\text{stmt}(\sigma, s) = \text{stmt}(\sigma, s)$$

6 Usage Examples

7 References

- Chalmers University of Technology. *QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs* <https://dl.acm.org/doi/epdf/10.1145/351240.351266>
- Hackage. *QuickCheck: Automatic testing of Haskell programs* <https://hackage.haskell.org/package/QuickCheck>
- sulzmann.github.io. *QuickCheck: Automatic Testing* <https://sulzmann.github.io/ProgrammingParadigms/lec-haskell-testing.html>
- University of Nottingham. *Programming language semantics: It's easy as 1,2,3* <https://people.cs.nott.ac.uk/pszgmh/123.pdf>)